AG TECHNOLOGIES

Service Gestion des Utilisateurs

Profils, Rôles & Permissions Dynamiques — User Service

Cahier des Charges Technique v2.1

implementation-ready

Version	2.1 (implementation-ready)
Date	Mars 2026
Statut	Post-audit — Corrections appliquées
Classification	Confidentiel
Auteur	Équipe Architecture

Document confidentiel — Réservé à l'usage interne d'AG Technologies

Historique des Révisions

Ver.	Date	Description
1.0	20/03/2026	Création initiale
2.0	20/03/2026	Post-audit : plateformes unifiées (Auth=maître, UUID), email/phone read-only, contrats inter-services, MLD corrigé, rate limiting, health check
2.1	20/03/2026	Clarifications finales : modèle de suppression dual (quitter plateforme vs soft delete général + deactivate Auth), séquence hard delete sécurisée, endpoint retrait plateforme, {id}=users_profiles.id explicité, endpoint lookup par auth_user_id, assigned_by clarifié

Contents

1 Introduction **5**
- 1.1 Objet . . . 5
- 1.2 Périmètre . . . 5
- 1.3 Destinataires . . . 5

2 Contexte & Problématique **5**

3 Vision & Objectifs **5**
- 3.1 KPIs . . . 5

4 Parties Prenantes **6**

5 Besoins Fonctionnels **6**
- 5.1 Module Profil . . . 6
- 5.2 Module Adresses . . . 6
- 5.3 Module RBAC Dynamique . . . 6
- 5.4 Module Documents . . . 6
- 5.5 Module Métadonnées . . . 6
- 5.6 Module Recherche & Statistiques . . . 7

6 Besoins Non-Fonctionnels **7**

7 Architecture Technique **7**
- 7.1 Stack . . . 7
- 7.2 Dépendances . . . 7
- 7.3 Modèle d'Autorisation . . . 7

8 Modélisation — MLD **8**
- 8.1 Table `users_profiles` . . . 8
- 8.2 Table `addresses` . . . 8
- 8.3 Table `roles` . . . 8
- 8.4 Table `permissions` . . . 8
- 8.5 Table `role_permissions` . . . 9
- 8.6 Table `user_roles` . . . 9
- 8.7 Table `documents` . . . 9
- 8.8 Table `document_history` . . . 9
- 8.9 Table `user_metadata` . . . 9
- 8.10 Table `audit_logs` . . . 10

9 Contrats d'API **11**
- 9.1 GET /api/v1/health . . . 11
- 9.2 Profil . . . 11
  - 9.2.1 POST /api/v1/users — Provisioning . . . 11
  - 9.2.2 POST /api/v1/users/sync — Sync email/phone . . . 11
  - 9.2.3 POST /api/v1/users/status-sync — Sync status . . . 11
  - 9.2.4 GET /api/v1/users/{id} . . . 11
  - 9.2.5 GET /api/v1/users/by-auth/{authUserId} — Lookup . . . 11
  - 9.2.6 PUT /api/v1/users/{id} . . . 11
  - 9.2.7 DELETE /api/v1/users/{id}/platforms/{platformId} — Quitter une plateforme 12
  - 9.2.8 DELETE /api/v1/users/{id} — Soft delete général . . . 12
  - 9.2.9 DELETE /api/v1/users/{id}/permanent — Hard delete . . . 12
  - 9.2.10 PUT /api/v1/users/{id}/photo . . . 12

9.2.11 GET /api/v1/users . . . 12
9.2.12 GET /api/v1/users/search . . . 12
9.2.13 GET /api/v1/users/stats . . . 12
9.3 Adresses . . . 12
9.3.1 POST /api/v1/users/{id}/addresses . . . 12
9.3.2 GET, PUT, DELETE — standard CRUD . . . 12
9.3.3 PUT /api/v1/users/{id}/addresses/{addressId}/default . . . 12
9.4 Rôles . . . 12
9.4.1 POST /api/v1/platforms/{platformId}/roles . . . 13
9.4.2 GET, PUT, DELETE — standard CRUD . . . 13
9.5 Permissions . . . 13
9.5.1 POST /api/v1/platforms/{platformId}/permissions . . . 13
9.5.2 POST /api/v1/platforms/{platformId}/roles/{roleId}/permissions . . . 13
9.5.3 DELETE .../permissions/{permId} . . . 13
9.6 Rôles Utilisateur . . . 13
9.6.1 POST /api/v1/users/{id}/roles . . . 13
9.6.2 GET /api/v1/users/{id}/roles . . . 13
9.6.3 DELETE /api/v1/users/{id}/roles/{roleId} . . . 13
9.6.4 GET /api/v1/users/{id}/permissions/check . . . 13
9.7 Documents . . . 13
9.7.1 POST /api/v1/users/{id}/documents . . . 13
9.7.2 GET /api/v1/users/{id}/documents . . . 13
9.7.3 PUT /api/v1/users/{id}/documents/{docId}/status . . . 13
9.7.4 DELETE, GET .../history — standard . . . 13
9.8 Métadonnées . . . 13
9.8.1 PUT /api/v1/users/{id}/metadata/{platformId} . . . 14
9.8.2 GET, DELETE .../{key} — standard . . . 14

10 Contrats Inter-Services **15**
10.1 Auth → Users (Provisioning) . . . 15
10.2 Auth → Users (Sync email/phone) . . . 15
10.3 Auth → Users (Sync status) . . . 15
10.4 Users → Auth (Purge RGPD) . . . 15
10.5 Users → Notification . . . 16
10.6 Users → Média . . . 16

11 Rate Limiting **16**

12 Infrastructure **16**

13 Mapping Statuts Auth ↔ Users **16**

14 Recommandations **17**

Introduction

Objet

Cahier des charges technique du User Service d'AG Technologies.

Périmètre

Inclus : profils, adresses, RBAC dynamique, documents, métadonnées, recherche, statistiques, audit trail.

Hors périmètre :

- Authentification, credentials, tokens → Service Auth
- Stockage fichiers → Service Média
- Envoi emails/SMS → Service Notification
- CRUD plateformes → Service Auth (registre maître)
- Logique métier / KYC des plateformes

Source de vérité : Auth est la source maître pour l'identité d'authentification (`auth_user_id`, email, phone, statut d'accès). Users est la source maître pour le profil, les rôles, permissions, adresses, documents et métadonnées.

Destinataires

Destinataire	Rôle
Équipe Backend	Développement et maintenance
Équipe Frontend	Intégration APIs
Équipes consommatrices	AGT-Market, AGT-Bot, SALMA, Auth
Lead Technique	Validation choix techniques

Contexte & Problématique

Problème	Solution
Profils dupliqués	Profil unique partagé
Rôles codés en dur	RBAC 100% dynamique via API
Données incohérentes	Source unique de vérité
Suppression complexe	Soft + hard delete RGPD configurable
Vérification droits	Endpoint rapide + cache Redis

Vision & Objectifs

Le Service Users est **générique, abstrait et dynamique**. Aucun concept métier codé en dur.

Principe : Users = registre central de *qui est qui* et *ce qu'il peut faire* — sans contexte métier.

KPIs

KPI	MVP	Long terme
Lecture profil	< 200ms	< 100ms
Vérification permission	< 200ms	< 50ms
Recherche	< 500ms	< 200ms
Disponibilité	$\geq 99\%$	$\geq 99{,}5\%$

Parties Prenantes

Service	Usage
AGT-Market	Rôles client/vendeur/livreur, documents KYC
SALMA	Rôles étudiant/formateur, documents diplômes
AGT-Bot	Rôles tenant/opérateur
Service Auth	POST /users après inscription (provisioning sync)
Service Notification	Alertes changements rôles, validation documents
Service Média	Stockage photos et documents

Besoins Fonctionnels

Module Profil

ID	Fonctionnalité	Priorité
F-PRO-01	Création profil lié à Auth (auth_user_id)	Urgent
F-PRO-02	Consultation profil	Urgent
F-PRO-03	MAJ nom, prénom, naissance, genre, photo	Urgent
F-PRO-04	Email/phone read-only sync Auth	Urgent
F-PRO-05	Quitter une plateforme (retrait rôles, métadonnées, documents)	Urgent
F-PRO-06	Soft delete général (status=deleted + deactivate Auth)	Urgent
F-PRO-07	Hard delete RGPD (délai configurable, séquence sécurisée)	Urgent
F-PRO-08	Upload photo via Média	Normal

Module Adresses

ID	Fonctionnalité	Priorité
F-ADR-01	CRUD adresses, type libre	Urgent
F-ADR-02	Adresse par défaut	Normal

Module RBAC Dynamique

Aucun rôle ni permission codé en dur. Tout est créé dynamiquement via l'API.

ID	Fonctionnalité	Priorité
F-ROL-01	CRUD rôles par plateforme	Urgent
F-ROL-02	CRUD permissions par plateforme	Urgent
F-ROL-03	Attacher/détacher permissions à un rôle	Urgent
F-ROL-04	Assigner/retirer rôles par user par plateforme	Urgent
F-ROL-05	Multi-rôles, multi-plateformes	Urgent
F-ROL-06	Vérification permission (rapide, cache Redis)	Urgent

Module Documents

ID	Fonctionnalité	Priorité
F-DOC-01	Attacher document, type libre	Urgent
F-DOC-02	Fichier dans Média, Users garde référence	Urgent
F-DOC-03	Workflow pending→validated/rejected	Urgent
F-DOC-04	Re-soumission (historique conservé)	Normal
F-DOC-05	Listage filtrable	Normal

Module Métadonnées

Système clé-valeur flexible par plateforme, sans modification du schéma central.

Module Recherche & Statistiques

Listage paginé (filtres plateforme, rôle, statut), recherche texte, stats par plateforme/rôle/statut.

Besoins Non-Fonctionnels

ID	Catégorie	Exigence	Cible
NF-01	Performance	Lecture profil	$<$ 200ms (P95)
NF-02	Performance	Vérif. permission	$<$ 200ms (cache)
NF-03	Performance	Recherche	$<$ 500ms
NF-04	Performance	Écriture	$<$ 300ms
NF-05	Disponibilité	Uptime	$\geq$ 99,5%
NF-06	Sécurité	Auth obligatoire	Bearer token
NF-07	Sécurité	Isolation données	Par plateforme
NF-08	Sécurité	PII protégés	Exclus des logs
NF-09	Sécurité	Audit trail	Toute modification
NF-10	Scalabilité	Horizontal	Stateless
NF-11	Scalabilité	Cache	Redis TTL config.
NF-12	Observabilité	Health check	GET /api/v1/health
NF-13	Observabilité	Logs	JSON structuré

Architecture Technique

Stack

Composant	Technologie
Langage & Framework	Python 3.11+ / Django 5.x
API	Django REST Framework
Base de données	PostgreSQL 15+
Cache	Redis 7+
Conteneurisation	Docker + Docker Compose
CI/CD	GitHub Actions

Dépendances

Règle : Users dépend de Auth. Auth ne dépend PAS de Users. Plateformes gérées par Auth (registre maître).

Service	Type	Usage
Service Auth	Interne	Validation JWT, source plateformes, source email/phone
Service Média	Interne	Stockage photos et documents
Service Notification	Interne	Alertes rôles, validation documents

Modèle d'Autorisation

Niveau	Mécanisme	Endpoints
Service-to-Service	Token S2S Auth	POST /users, purge RGPD
Admin plateforme	Bearer + RBAC (manage_roles...)	CRUD rôles, permissions, documents
Utilisateur	Bearer token	Son profil, adresses, documents

Chaque plateforme crée ses rôles admin à l'init avec permissions nécessaires. Pas de mécanisme spécial.

Modélisation — MLD

v2.0 : Plus de table `platforms` locale. Tous les `platform_id` = UUID Auth.

Table `users_profiles`

Colonne	Type	Contrainte	Description
id	UUID	PK	Identifiant
auth_user_id	UUID	NOT NULL, UNIQUE	FK logique → Auth
first_name	VARCHAR(100)	NOT NULL	Prénom
last_name	VARCHAR(100)	NOT NULL	Nom
email	VARCHAR(255)	NULL	Read-only (Auth)
phone	VARCHAR(20)	NULL	Read-only (Auth)
avatar_url	VARCHAR(500)	NULL	URL photo
birth_date	DATE	NULL	Naissance
gender	VARCHAR(20)	NULL	Genre
status	VARCHAR(20)	DEFAULT 'active'	active/inactive/deleted/deletion_in_progress
deleted_at	TIMESTAMPTZ	NULL	Soft delete
hard_delete_after	TIMESTAMPTZ	NULL	Hard delete planifié
purge_auth_pending	BOOLEAN	DEFAULT false	Purge Auth en attente après échec
deletion_error_reason	TEXT	NULL	Dernier motif d'échec suppression
created_at	TIMESTAMPTZ	NOT NULL	Création
updated_at	TIMESTAMPTZ	NOT NULL	MAJ

Table `addresses`

Colonne	Type	Contrainte	Description
id	UUID	PK	
user_id	UUID	FK → users_profiles	
type	VARCHAR(50)	NOT NULL	Type libre
street	VARCHAR(255)	NOT NULL	
city	VARCHAR(100)	NOT NULL	
country	VARCHAR(100)	NOT NULL	
postal_code	VARCHAR(20)	NULL	Code postal
is_default	BOOLEAN	DEFAULT false	
created_at	TIMESTAMPTZ	NOT NULL	
updated_at	TIMESTAMPTZ	NOT NULL	

Table `roles`

Colonne	Type	Contrainte	Description
id	UUID	PK	
platform_id	UUID	NOT NULL	Réf. Auth
name	VARCHAR(100)	NOT NULL	
description	TEXT	NULL	
created_at	TIMESTAMPTZ	NOT NULL	

UNIQUE : (platform_id, name)

Table `permissions`

Colonne	Type	Contrainte	Description
id	UUID	PK	
platform_id	UUID	NOT NULL	Réf. Auth
name	VARCHAR(100)	NOT NULL	
description	TEXT	NULL	
created_at	TIMESTAMPTZ	NOT NULL	

UNIQUE : (platform_id, name)

Table `role_permissions`

PK composite (role_id, permission_id). FK → roles, FK → permissions. + created_at.

Table `user_roles`

Colonne	Type	Contrainte	Description
id	UUID	PK	
user_id	UUID	FK → users_profiles	
role_id	UUID	FK → roles	
assigned_by	UUID	NULL	Admin
assigned_at	TIMESTAMPTZ	NOT NULL	

UNIQUE : (user_id, role_id). Note v2.0 : platform_id retiré (redondant). **Note v2.1 :** `assigned_by` = `users_profiles.id` de l'admin qui a assigné le rôle (cohérent avec audit_logs.actor_id).

Table `documents`

Colonne	Type	Contrainte	Description
id	UUID	PK	
user_id	UUID	FK → users_profiles	
platform_id	UUID	NOT NULL	Plateforme
doc_type	VARCHAR(100)	NOT NULL	Type libre
media_id	UUID	NOT NULL	Réf. Média
status	VARCHAR(20)	DEFAULT 'pending'	
comment	TEXT	NULL	Review
submitted_at	TIMESTAMPTZ	NOT NULL	
reviewed_at	TIMESTAMPTZ	NULL	
created_at	TIMESTAMPTZ	NOT NULL	

Table `document_history`

Colonne	Type	Contrainte	Description
id	UUID	PK	
document_id	UUID	FK → documents	Parent
media_id	UUID	NOT NULL	Ancienne réf.
status	VARCHAR(20)	NOT NULL	Archivé
comment	TEXT	NULL	
submitted_at	TIMESTAMPTZ	NOT NULL	
reviewed_at	TIMESTAMPTZ	NULL	
archived_at	TIMESTAMPTZ	NOT NULL	

Table `user_metadata`

Colonne	Type	Contrainte	Description
id	UUID	PK	
user_id	UUID	FK → users_profiles	
platform_id	UUID	NOT NULL	
key	VARCHAR(100)	NOT NULL	
value	TEXT	NOT NULL	
updated_at	TIMESTAMPTZ	NOT NULL	

UNIQUE : (user_id, platform_id, key)

Table `audit_logs`

Colonne	Type	Contrainte	Description
id	UUID	PK	
entity_type	VARCHAR(50)	NOT NULL	Table modifiée
entity_id	UUID	NOT NULL	
action	VARCHAR(30)	NOT NULL	create, update...
actor_id	UUID	NULL	NULL=système
actor_type	VARCHAR(20)	DEFAULT 'user'	user/service/system
old_value	JSONB	NULL	
new_value	JSONB	NULL	
created_at	TIMESTAMPTZ	NOT NULL	

Index : (entity_type, entity_id), (actor_id), (created_at)

Contrats d'API

Base URL : `/api/v1` **Format :** JSON **Auth :** Bearer token (sauf mention)
Pagination :

```
{"data": [...], "page": 1, "limit": 20, "total": 142}
```

Convention v2.1 : dans tous les endpoints `/users/{id}`, `{id}` désigne toujours `users_profiles.id` (UUID interne Users). Les services qui ne disposent que du `auth_user_id` doivent d'abord appeler `GET /api/v1/users/by-auth/{authUserId}` pour résoudre l'ID.

GET /api/v1/health

Sans auth. Vérifie DB + Redis.

```
{"status":"healthy","database":"ok","redis":"ok","version":"2.1"}
```

Profil

POST /api/v1/users — Provisioning

Auth : Token S2S (par Auth après inscription).

```
{"auth_user_id":"uuid","first_name":"Jean","last_name":"Dupont",
 "email":"jean@example.com","phone":"+237600000000"}
```

Note v2.1 : les champs email/phone sont une **projection initiale** depuis Auth. Users ne les modifie jamais lui-même — seul Auth peut les mettre à jour via POST /api/v1/users/sync.
Codes : 201, 400, 409 (existe), 401.

POST /api/v1/users/sync — Sync email/phone

Auth : Token S2S. Par Auth quand email/phone change.

```
{"auth_user_id":"uuid","email":"new@example.com","phone":"+237611111111"}
```

Codes : 200, 404, 401.

POST /api/v1/users/status-sync — Sync status

Auth : Token S2S. Utilisé par Auth pour propager un changement d'état de compte vers Users (ex: désactivation).

```
{"auth_user_id":"uuid","status":"inactive"}
```

Codes : 200, 404, 401.

GET /api/v1/users/{id}

Codes : 200, 401, 403, 404.

GET /api/v1/users/by-auth/{authUserId} — Lookup

Auth : Bearer ou Token S2S. Résout un `auth_user_id` vers un profil Users complet. Utile pour les intégrations inter-services.
Codes : 200, 401, 404.

PUT /api/v1/users/{id}

Auth : propriétaire ou manage_users. Champs : first_name, last_name, birth_date, gender. **email/-phone NON modifiables.**

DELETE /api/v1/users/{id}/platforms/{platformId} — Quitter une plateforme

Auth : Bearer (propriétaire) ou manage_users. Retire tous les rôles et métadonnées de l'utilisateur sur cette plateforme, et archive logiquement les documents associés sans suppression immédiate des fichiers physiques. Le profil global et le compte Auth restent intacts. L'utilisateur peut revenir.

```
{"message":"Platform left","platform_id":"uuid",
 "roles_removed":2,"metadata_cleared":true,"documents_archived":3}
```

Codes : 200, 401, 403, 404.

DELETE /api/v1/users/{id} — Soft delete général

Auth : Bearer (propriétaire). Suppression complète du compte sur **toutes les plateformes**. Marque status=deleted, calcule hard_delete_after. **Déclenche aussi une désactivation Auth via un endpoint S2S dédié (sans mot de passe), afin de bloquer immédiatement la connexion utilisateur.**

```
{"message":"Account deactivated","status":"deleted",
 "hard_delete_scheduled":"2026-04-19T10:00:00Z"}
```

Codes : 200, 401, 403, 404.

Modèle de suppression dual (v2.1) :
— **Quitter une plateforme** = retrait rôles/métadonnées/documents de cette plateforme. Profil et Auth intacts.
— **Soft delete général** = profil marqué deleted + Auth désactivé. Hard delete après délai configurable.
Un utilisateur multi-plateforme peut quitter une plateforme sans perdre son compte global.

DELETE /api/v1/users/{id}/permanent — Hard delete

Auth : manage_users ou S2S. Irréversible + purge Auth.

PUT /api/v1/users/{id}/photo

Body : `{"media_id":"uuid"}`

GET /api/v1/users

Auth : manage_users. Query : platform_id, role, status, page, limit.

GET /api/v1/users/search

Auth : manage_users. Query : q, page, limit.

GET /api/v1/users/stats

Auth : view_stats.

Adresses

Auth : propriétaire ou manage_users.

POST /api/v1/users/{id}/addresses

```
{"type":"livraison","street":"123 Rue","city":"Yaounde",
 "country":"Cameroun","postal_code":"BP1234","is_default":true}
```

GET, PUT, DELETE — standard CRUD

PUT /api/v1/users/{id}/addresses/{addressId}/default

Rôles

Auth : manage_roles.

POST /api/v1/platforms/{platformId}/roles

```
{"name":"vendeur","description":"Utilisateur vendeur"}
```

platformId = UUID Auth. Codes : 201, 400, 403, 409.

GET, PUT, DELETE — standard CRUD

Permissions

Auth : manage_roles.

POST /api/v1/platforms/{platformId}/permissions

```
{"name":"create_product","description":"Peut creer des produits"}
```

POST /api/v1/platforms/{platformId}/roles/{roleId}/permissions

Body : `{"permission_id":"uuid"}`

DELETE .../permissions/{permId}

Rôles Utilisateur

Auth : manage_roles.

POST /api/v1/users/{id}/roles

Body : `{"role_id":"uuid"}`. Codes : 201, 409 (déjà assigné).

GET /api/v1/users/{id}/roles

Query : ?platform_id=uuid.

```
{"user_id":"uuid","roles":[
  {"role":{"id":"...","name":"vendeur"},"platform_id":"uuid",
   "assigned_at":"...","assigned_by":"uuid"}]}
```

DELETE /api/v1/users/{id}/roles/{roleId}

GET /api/v1/users/{id}/permissions/check

Auth : Bearer ou S2S. Cache Redis. Query : platform_id, permission (obligatoires).

```
{"user_id":"uuid","platform_id":"uuid","permission":"create_product",
 "granted":true,"via_role":"vendeur"}
```

Cache : `perm:{userId}:{platformId}:{permName}`, TTL config. Invalidation auto.

Documents

POST /api/v1/users/{id}/documents

```
{"platform_id":"uuid","doc_type":"cni","media_id":"uuid"}
```

Re-soumission : ancien archivé dans document_history.

GET /api/v1/users/{id}/documents

Query : ?type=cni&status=pending

PUT /api/v1/users/{id}/documents/{docId}/status

Auth : manage_documents. Body : `{"status":"validated","comment":"OK"}`

DELETE, GET .../history — standard

Métadonnées

PUT /api/v1/users/{id}/metadata/{platformId}

```
{"nom_boutique":"Tech Shop","siret":"123456789"}
```

Upsert.

GET, DELETE .../{key} — standard

Contrats Inter-Services

Auth → Users (Provisioning)

Param.	Valeur
Déclencheur	Register réussi (email, phone, OAuth)
Transport	HTTP POST sync
Endpoint	POST /api/v1/users
Auth	Token S2S
Timeout/Retry	5s / 3x backoff (1s,2s,4s)
Échec	Auth crée compte + warning, cron retente

Auth → Users (Sync email/phone)

Param.	Valeur
Déclencheur	Changement email/phone dans Auth
Transport	HTTP POST sync
Endpoint	POST /api/v1/users/sync
Échec	Auth logge, opération validée

Auth → Users (Sync status)

Param.	Valeur
Déclencheur	Changement d'état du compte dans Auth (`is_deactivated`)
Transport	HTTP POST sync
Endpoint	POST /api/v1/users/status-sync
Auth	Token S2S
Payload	`{"auth_user_id":"uuid","status":"inactive"}`
Échec	Auth logge, opération Auth reste valide

Users → Auth (Purge RGPD)

Contrat additionnel v2.1 — Soft delete général : quand un utilisateur soft-delete son compte global, Users appelle un endpoint Auth **S2S dédié** de désactivation propagée. **L'endpoint utilisateur** `POST /auth/account/deactivate` reste réservé aux appels frontend avec mot de passe. Une extension Auth v1.4 est requise pour la propagation inter-service.

Param.	Valeur
Déclencheur	DELETE /api/v1/users/{id} (soft delete global)
Transport	HTTP POST sync
Endpoint Auth	POST /auth/admin/deactivate/{authUserId} (Auth v1.4)
Auth	Token S2S ou X-Admin-API-Key
Effet	Positionne `is_deactivated=true`, révoque sessions et refresh tokens
Échec	Users annule ou journalise pour retry

Param.	Valeur
Déclencheur	Cron: hard_delete_after dépassé, ou admin
Transport	HTTP DELETE sync
Endpoint Auth	DELETE /auth/admin/purge/{userId} (Auth v1.4)
Séquence	1) Marquer status=deletion_in_progress 2) Appel Auth purge 3) Si succès: purge Users 4) Si échec: purge_auth_pending=true, retry cron
Échec	Données Users intactes tant qu'Auth n'a pas confirmé

Auth v1.4 requis : DELETE /auth/admin/purge/{userId} — supprime users_auth, sessions, refresh_tokens, oauth_providers, login_history, verification_tokens.

Users → Notification

Transport : HTTP POST sync (MVP). Token S2S. Timeout 5s, 3x retry.

Types : role_assigned, role_removed, document_validated, document_rejected, account_deleted.

Users → Média

Frontend upload direct vers Média, Users reçoit media_id. Users supprime via Média au hard delete. Auth : S2S.

Rate Limiting

Endpoint	Fen.	Limite	Strat.
/users/search	1 min	20	IP+user
/users/{id}/permissions/check	1 min	100	IP
POST .../documents	1h	10	user
Global	1 min	200	IP

Redis sliding window.

Infrastructure

Variable	Description
DATABASE_URL	PostgreSQL
REDIS_URL	Redis
AUTH_SERVICE_URL	Service Auth
MEDIA_SERVICE_URL	Service Média
NOTIFICATION_SERVICE_URL	Service Notification
PERMISSION_CACHE_TTL	Cache permissions (300s)
PROFILE_CACHE_TTL	Cache profils (600s)
SECRET_KEY	Django secret
DEFAULT_HARD_DELETE_DELAY_DAYS	Délai RGPD (30j)
PURGE_CRON_SCHEDULE	Cron purge (0 2 * * *)

Mapping Statuts Auth ↔ Users

Événement	Auth	Users	Propagation
Inscription	Compte	Profil active	Auth→POST /users
Blocage	is_blocked	Pas d'impact	Aucune
Désactivation Auth	is_deactivated	status=inactive	Auth→POST /users/status-sync
Quitter plateforme	—	Rôles/méta/docs retirés	Users seul
Soft delete global	is_deactivated	status=deleted	Users→Auth S2S deactivate
Hard delete	Purge creds	Purge profil	Users→Auth purge

Auth gère l'accès et l'état d'authentification. Users gère le cycle de vie du profil et l'appartenance aux plateformes. Toute propagation de statut passe par un contrat inter-service explicite. Pas de mapping automatique `is_blocked` ↔ `status`.

Recommandations

- Migration SALMA : export → transform → POST /api/v1/users + rôles
- Cache permissions : invalidation auto sur modif rôle/assignation
- PII exclus des logs et réponses d'erreur
- Migration future vers event bus (RabbitMQ/Redis Streams)
- Monitoring Prometheus (requêtes, latence P95, cache hit ratio)
- Versioning : /api/v1, cohabitation /api/v2 si breaking changes

Glossaire

Terme	Définition
RBAC	Role-Based Access Control
Soft Delete	Suppression logique
Hard Delete	Suppression physique (RGPD)
PII	Personally Identifiable Information
JWT	JSON Web Token
S2S	Service-to-Service
Token S2S	Via POST /auth/s2s/token
TTL	Time To Live (cache)
Audit Trail	Historique des modifications

AG Technologies — Confidentiel — User Service CDC v2.1 — Mars 2026